```
Finished in 0.02645 seconds (files took 0.14491 seconds to load)
4 examples, 0 failures, 3 pending

Randomized with seed 23924
```

The API is now correctly recording the expense and returning the result we expect.

Now that we know the app is returning the expense ID correctly, let's move on to the next bit of behavior: rendering the correct HTTP status code. Fill in the body of the 'responds with a 200 (OK)' spec as follows:

05-unit-specs/04/expense_tracker/spec/unit/app/api_spec.rb

```
it 'responds with a 200 (OK)' do
  expense = { 'some' => 'data' }

  allow(ledger).to receive(:record)
    .with(expense)
    .and_return(RecordResult.new(true, 417, nil))

  post '/expenses', JSON.generate(expense)
  expect(last_response.status).to eq(200)
end
```

Go ahead and run this spec file:

```
$ bundle exec rspec spec/unit/app/api_spec.rb

Randomized with seed 55289

ExpenseTracker::API
  POST /expenses
    when the expense is successfully recorded
      returns the expense id
      responds with a 200 (OK)
    when the expense fails validation
      responds with a 422 (Unprocessable entity) (PENDING: Not yet
      implemented)
      returns an error message (PENDING: Not yet implemented)

« truncated »

Finished in 0.02565 seconds (files took 0.12232 seconds to load)
4 examples, 0 failures, 2 pending

Randomized with seed 55289
```

It passes, because Sinatra returns a 200 HTTP status code unless an error occurs or you explicitly set one. That should make you wonder whether or not the test actually works. Let's temporarily break the app code to make sure the test will catch it: